

CODETRACE AI — Full Platform Report

"Beyond Matching. Understand the Code."

Problem Statement: RAMPeX Hackathon Problem Statement 05 — AI-Powered Code Integrity & Plagiarism Intelligence

Design Principle: "Simple for the user. Powerful underneath."

Core Innovation: Multi-signal code intelligence combining Python AST canonical identifier renaming, 3-gram token chain alignment, local TF-IDF vector space semantic modeling, behavioral submission timing, and explainable natural language evidence cards.

1. Executive Dashboard & Integrity Overview

24 Submissions	6 Need Review	18 Students	82% Avg. Confidence
-------------------	------------------	----------------	------------------------

Requires Attention — Flagged Priority Pairs

Student Pair	Assignment	Similarity	Plain-Language Reason	Action
Charlie Davis ↔ Diana Evans	HW3: Algorithms	100%	Strong structural + semantic overlap despite 100% identifier renaming	Review Needed
George Clark ↔ Hannah Abbott	HW3: Algorithms	91%	Reordered independent variable assignment & nested control flow	Review Needed
Kevin Bacon ↔ Laura Croft	HW3: Algorithms	82%	Copied core binary search loop with added print statements	Review Needed
Alice Chen ↔ Bob Smith	HW3: Algorithms	66%	Common hash map logic with distinct loop structure	Moderate Risk

2. Guided 4-Step Upload & Analysis Wizard

Step 1 — Select Assignment: Select homework target (e.g. CS101 — HW3: Algorithm Optimization)

Step 2 — Student Details: Enter student name and institutional ID tag (e.g. Maya Lin — STU-2099)

Step 3 — Language & Code: Select programming language (Python, JavaScript, Java, C++) & paste source code

Step 4 — Automated Analysis: Triggers real-time AST normalization, token sequence extraction, and multi-signal comparison.

Stage	Analysis Subsystem	Status
1/5	Normalizing code (removing comments & variable noise)	Completed
2/5	Extracting AST structural patterns & node depth	Completed
3/5	Comparing pairwise submissions	Completed
4/5	Generating explainable evidence cards	Completed
5/5	Complete! Results rendered to dashboard	Completed

3. Side-by-Side Code Comparison & Explainable Evidence

Case Study: Charlie Davis (STU-1003) vs Diana Evans (STU-1004) — Similarity: 100%

Source Code Comparison (Original View)

<pre># Student A: Charlie Davis def calculate_total_tax(items, tax_rate): subtotal = 0 for item in items: subtotal += item['price'] * item['quantity'] total_tax = subtotal * tax_rate final_amount = subtotal + total_tax return final_amount</pre>	<pre># Student B: Diana Evans def compute_final_price(cart, vat_ratio): accumulated_cost = 0 for element in cart: accumulated_cost += element['price'] * ele tax_value = accumulated_cost * vat_ratio net_cost = accumulated_cost + tax_value return net_cost</pre>
--	---

AST Normalized View (Variable Renaming Bypassed)

```
def func_1(v_1, v_2):
    v_3 = 0
    for v_4 in v_1:
        v_3 += v_4['price'] * v_4['quantity']
    v_5 = v_3 * v_2
    v_6 = v_3 + v_5
    return v_6
```

- 1. Code Structure (100% AST Match): Both solutions exhibit an identical control-flow architecture (For -> Assign -> Return). The nested loop and accumulator profiles match exactly.
- 2. Code Meaning (100% Semantic Overlap): Variable names differ (subtotal vs accumulated_cost), but after canonical AST normalization, statement execution vectors yield 100% semantic overlap.
- 3. Token Patterns (100% Token Match): After stripping comments and normalizing identifiers, 3-gram token sequence analysis yields exact chain alignment.
- 4. Temporal Proximity: Submissions were uploaded within 3 minutes of each other.

4. Cross-Student Similarity Matrix & Network Graph

Pairwise Cross-Student Heatmap Grid

Student	Alice	Bob	Charlie	Diana	George	Hannah
Alice	—	66%	18%	19%	24%	22%
Bob	66%	—	21%	20%	25%	23%
Charlie	18%	21%	—	100%	31%	30%
Diana	19%	20%	100%	—	29%	28%
George	24%	25%	31%	29%	—	91%
Hannah	22%	23%	30%	28%	91%	—

5. Programmatic Benchmark Evaluation Suite

Comparison of CodeTrace AI Multi-Signal Engine vs Naive Text Matching over N=6 Synthetic Test Cases

Metric	CodeTrace AI (Multi-Signal)	Naive Text Matching	Performance Delta
Accuracy	100.0%	50.0%	+50.0% Improvement
Precision	100.0%	100.0%	Parity (1.00)
Recall (Detection Rate)	100.0%	25.0%	+75.0% Detection Gain
F1 Score	100.0%	40.0%	+60.0% Overall Score
False Positive Rate	0.0%	0.0%	0.0% False Alarms

6. Synthetic Benchmark Test Dataset Cases (N=6)

- Case 1 — Original Solutions: Two Sum Hash Map vs Brute Force → Expected: Low Similarity (Passed)
- Case 2 — Variable Renaming: Identical logic, renamed identifiers → Expected: Review Recommended (Passed)
- Case 3 — Formatting & Comments: Docstring additions & re-indentation → Expected: Review Recommended (Passed)
- Case 4 — Statement Reordering: Independent assignment reordering → Expected: High Similarity (Passed)
- Case 5 — Algorithm Variants: Recursive vs Iterative Factorial → Expected: Low Similarity (Passed)
- Case 6 — Modified Copy: Copied loop + extra print statement noise → Expected: High Similarity (Passed)

7. Admin Configuration & Scoring Formula

Mathematical Risk Aggregation Formula:

$Risk_Score = (20\% \times Token) + (30\% \times AST_Struct) + (30\% \times Semantic) + (15\% \times Logic) + (5\% \times Behavioral)$

- Classification Bounds:
- 0 – 29%: Low Similarity (Safe)
 - 30 – 59%: Moderate Similarity
 - 60 – 79%: High Similarity
 - 80 – 100%: Manual Review Recommended

